MinT: Managed Infrastructure for Training and Serving Millions of LLMs

Mind Lab

ABSTRACT

We present MindLab Toolkit (MinT), a managed infrastructure system for Low-Rank Adaptation (LoRA) post-training and online serving. MinT targets a setting where many trained policies are produced over a small number of expensive base-model deployments. Instead of materializing each policy as a merged full checkpoint, MinT keeps the base model resident and moves exported LoRA adapter revisions through rollout, update, export, evaluation, serving, and rollback. This adapter-revision path hides the complexity of distributed training, serving, scheduling, and data movement behind a service interface, making large-scale LoRA RL easier to run and reproduce.

MinT scales this adapter-revision path along three axes. Scale Up *extends LoRA RL to frontier-scale dense and Mixture-of-Experts (MoE) architectures, including MLA and DSA attention paths via LoRA target mapping and rollout correction, while model-parallel training and serving paths are validated beyond 1T total parameters.* Scale Down *minimizes the training-serving handoff by moving only the exported LoRA adapter, which can be less than 1% of the base-model size in compact rank-1 settings, eliminating full-checkpoint materialization entirely. In our measurements, adapter-only handoff reduces the measured handoff step by* 18.3× *on a 4B dense model and by* 2.85× *on a 30B MoE model; under the same resident-base allocation, concurrent multi-policy Group Relative Policy Optimization (GRPO) shortens wall time by* 1.77× *and* 1.45×*, respectively, without increasing peak memory.* Scale Out *expands the policy namespace while keeping engine-local execution bounded. MinT separates durable policy addressability from CPU/GPU hot working sets: a tensor-parallel serving deployment supports* 10^6*-scale addressable policy catalogs, with measured single-engine sweeps through 100K entries and thousand-adapter active waves at cluster scale. To address the main bottleneck that first-touch adapters serialize through the engine load path, MinT treats cold loading as scheduled service work and uses packed MoE LoRA tensors to improve live engine loading by* 8.5–8.7×*.Together, MinT makes multi-tenant training services a practical reality. Experimental validation demonstrates the infrastructure's ability to manage million-scale LoRA policy catalogs while training and serving selected adapter revisions over shared 1T-class base models.*

1 Introduction

Post-training has evolved from a simple one-pass stage in LLM production into a widely used infrastructure workload. As LLMs move toward trillion-scale parameter counts, real-world deployment, and lifelong learning from experience [45, 51], frontier model developers increasingly emphasize the complexity of building reliable frameworks for training modern agentic LLM capabilities [2, 8, 9, 14, 19, 28, 33, 36]. These workloads introduce a range of infrastructure challenges, including hardware resource management, distributed-computing scheduling, and version control for model artifacts. Traditional infrastructures rely on copying or serving a full fine-tuned checkpoint for each model variant are increasingly difficult to scale under the modern demands of continuous training, deployment and agentic reinforcement learning.

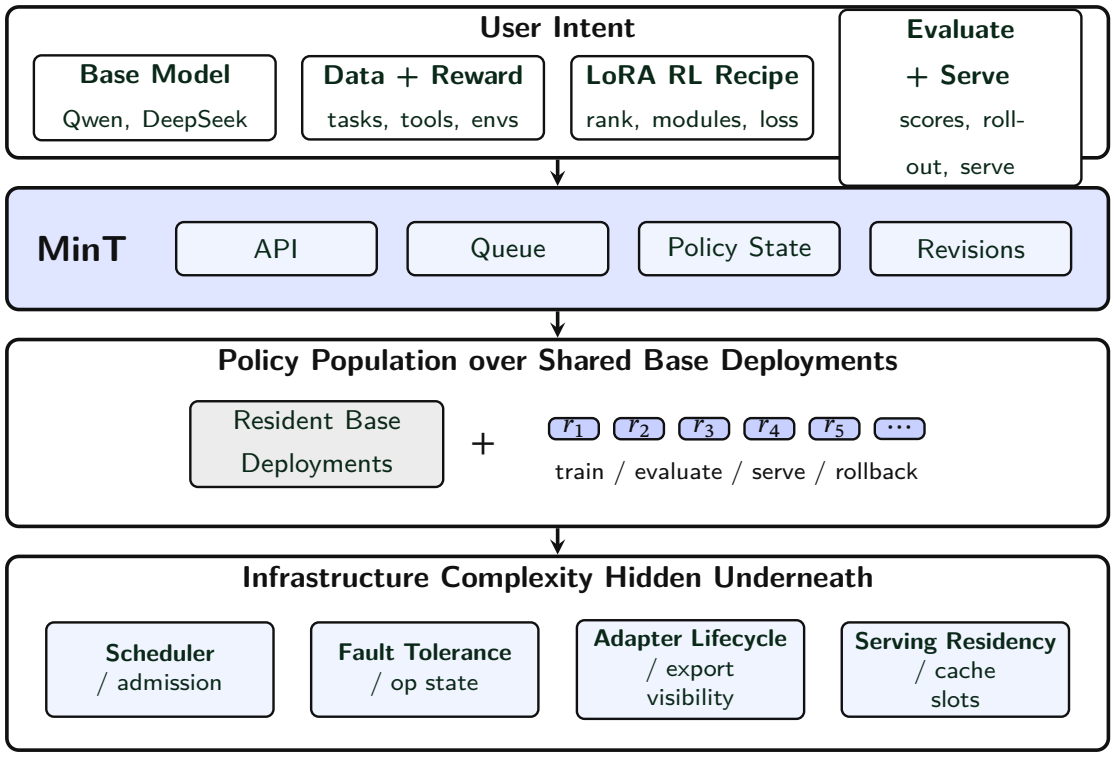


FIGURE 1 MinT overview. User intent selects a base model, data and rewards, a LoRA RL recipe, and evaluation or serving targets. MinT turns those inputs into queued work, policy records, and exported revisions, then manages a policy population over shared base deployments while scheduler, fault-tolerance, adapter-lifecycle, and serving-residency mechanisms remain behind the service interface.

To address these challenges, we propose the MindLab Toolkit (MinT), which uses Low-Rank Adaptation (LoRA) adapters as the basic policy units for post-training and online serving. In MinT, the base model remains resident, while task behaviors, product branches,

experimental versions, tenant-specific variants, and rollback points are represented by different adapters [16, 39]. As shown in Figure 1, MinT transforms user intent into queued jobs, policy records, and exported adapter revisions on top of shared base-model deployments. Instead of repeatedly moving, loading, or copying full models between training and serving, the system transfers and manages only compact LoRA parameters. Following the service-interface practice of Tinker [46, 47], MinT connects rollout, update, export, evaluation, serving, and rollback into a LoRA-centered workflow. It hides distributed training, model sharding, tensor-layout conversion, adapter admission, and serving access behind a simple service interface, making large-scale LoRA-based reinforcement learning easier to run, reproduce, and deploy.

This adapter-centered design changes what crosses the training-serving boundary. Full fine-tuning moves a full checkpoint for each trained variant. Merge-based LoRA reduces training memory, but still folds the adapter back into the base model and moves a merged checkpoint before inference. MinT instead exports the updated LoRA as a serving-compatible adapter revision, checks that it matches the resident base model, and loads it into an inference engine that already holds that base. Figure 2 illustrates this difference: MinT moves adapter revisions, not full model checkpoints.

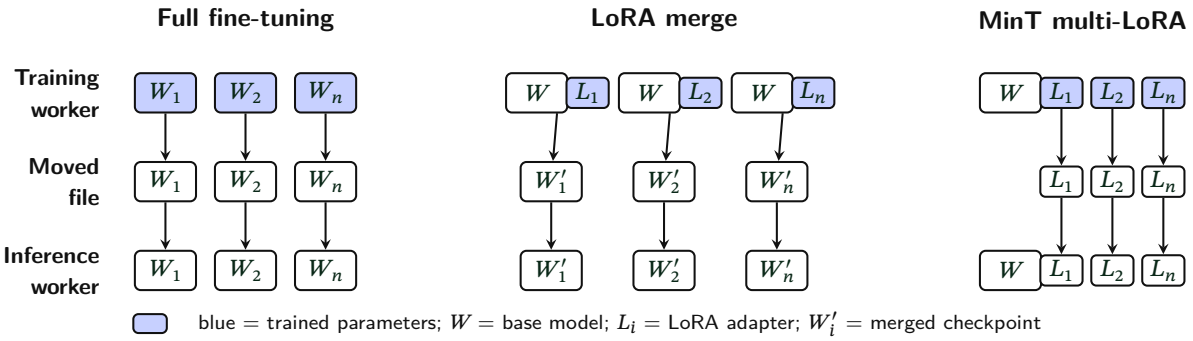


FIGURE 2 What crosses from training to serving under three adaptation paths. Blue marks the trained parameters. Full fine-tuning moves a full checkpoint for each variant. Merge-based LoRA trains adapters beside a resident base, then moves merged full checkpoints. MinT trains adapters beside a resident base and moves only adapter revisions to an inference engine that already holds the compatible base.

In this report, we introduce MinT's capabilities follow three scaling axes. *Scale Up* supports LoRA RL on frontier-scale dense and MoE architectures, with model-parallel training and serving paths validated beyond 1T total parameters [9, 14, 29, 35]. This axis keeps the same adapter-revision path usable when the base requires tensor-parallel or expert-parallel placement, sparse-route consistency, and distributed adapter export. *Scale Down* minimizes the training-serving handoff by moving only the exported LoRA adapter, which can be less than 1% of the base-model size in compact rank-1 settings, eliminating full-checkpoint materialization entirely. In our measurements, adapter-only handoff reduces

the measured handoff step by 18.3× on a 4B dense model and by 2.85× on a 30B MoE model; under the same resident-base allocation, concurrent multi-policy GRPO shortens wall time by 1.77× and 1.45×, respectively, without increasing peak memory. *Scale Out* expands the policy namespace while keeping engine-local execution bounded. MinT separates durable policy addressability from CPU/GPU hot working sets, builds on modern multi-adapter serving support [5, 18, 20, 43, 48], and treats first-touch adapter loading as scheduled service work. A tensor-parallel serving deployment supports 10^6-scale addressable policy catalogs, with measured single-engine sweeps through 100K entries and thousand-adapter active waves at cluster scale; packed MoE LoRA tensors improve live engine loading by 8.5–8.7× by reducing small-object fanout on the cold-load path. Together, these axes make continuously evolving LoRA post-training cheaper to operate, easier to reproduce, and better matched to multi-tenant policy populations.

MinT contributes four concrete pieces:

- Adapter lifecycle from update to serving. MinT exports trained LoRAs as PEFT adapter revisions, including sharded-to-serving conversion, compatibility checks, rollout records, sampler loading, served-result attribution, and rollback. The adapter revision becomes the fixed behavior selected by rollout, evaluation, online serving, and recovery.
- Large-scale multi-LoRA RL training. MinT time-slices LoRA training sessions over resident shared bases and supports both single-worker PEFT and distributed Megatron training paths. It reports end-to-end LoRA RL on large dense and MoE deployments, including 235B-A22B-scale training and a 1T-class MoE countdown-task path.
- Policy-population multi-LoRA serving. MinT serves exported adapters through shared-base vLLM engines, separates durable policy addressability from CPU/GPU hot working sets, and treats cold loading as scheduled work for cache misses. Its packed MoE LoRA serving representation reduces small-object fanout and speeds live engine loading by 8.5–8.7×.
- Public reproducibility paths. MinT provides a Tinker-compatible API [46, 47] and uses mint-cookbook recipes [27] to reproduce SFT, preference optimization, rollout-based RL, and AutoResearch examples through the same adapter lifecycle.

2 From LoRA Adapters to Managed Policy Revisions

Once many trained behaviors share a few resident base models, MinT has to separate the bytes that execute a behavior from the service state that makes the behavior manageable. An *adapter revision* is a fixed, exported snapshot of the LoRA adapter, frozen at a specific training step and stored in serving tensor layout; it is the executable LoRA payload that crosses from training to rollout, evaluation, and serving (it is not a training iteration or a transfer event). The policy record is the service-owned lifecycle state that makes that payload reproducible, reloadable, and rollbackable. This distinction lets a shared base

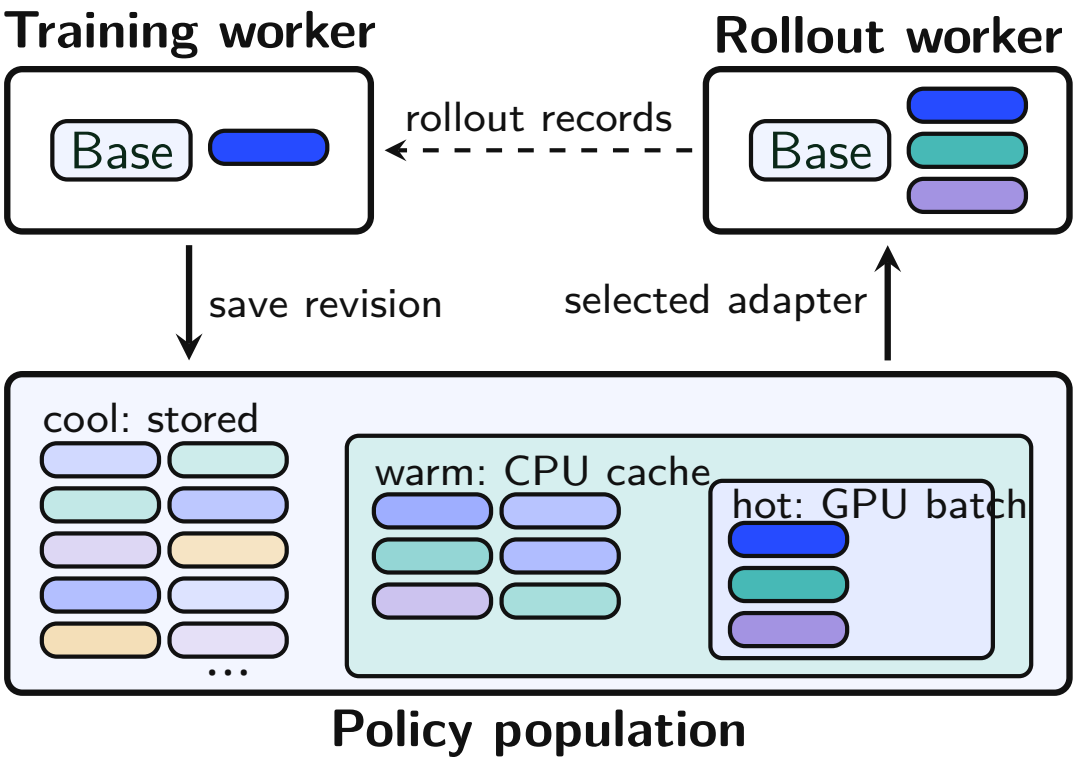


FIGURE 3 Adapter lifecycle in MinT. Training saves revisions into the policy population; rollout uses the hot GPU-batch subset and returns records for the next update.

support many LoRA policies without turning every policy into another full checkpoint or another full-model server.

An RL update leaves more than a LoRA file. After one update, the trainer has adapter tensors, optimizer moments, scheduler position, accumulated gradients, and rollout records that may still be needed for scoring. A sampler cannot consume that training checkpoint directly. It needs a fixed adapter revision in serving tensor layout, tied to the base model that is already resident in the inference engine. Later requests may select the same revision after the original worker has exited or after the serving cache has evicted the adapter bytes.

The service therefore has to answer four concrete questions after every update. Which base deployment can run this adapter? Which training checkpoint should a trainer restore if training resumes? Which fixed adapter revision should rollout, evaluation, or serving use? Where are those adapter bytes now: active in a GPU batch, cached in CPU memory, or only present in shared storage? MinT stores these answers separately because they change at different times.

A MinT policy record is the durable entry for one trained behavior over a compatible base model. It names the base version, LoRA rank and target modules, the latest training checkpoint, the rollout records kept for later updates, and the exported adapter revisions available for fixed behavior. A live training session is a temporary restoration of that record

on a worker. MinT allows one training session per policy record; concurrent branches become separate records so each worker writes a different adapter and optimizer state. Export freezes the current training checkpoint into a fixed adapter revision. Evaluation, rollout, and online serving select a revision, while worker placement and cache state can change around it.

The RL loop crosses this lifecycle on every iteration. Rollout samples trajectories with a selected adapter revision on an inference engine that already holds the base. Training recomputes token probabilities for those trajectories, applies the objective, and changes the adapter and optimizer state. The next rollout needs another export because inference engines consume fixed PEFT adapter files in serving layout. Trainer-local optimizer checkpoints stay on the training side. Evaluation uses the same fixed-revision rule so a reported score names the adapter revision that produced it.

Serving adds another time scale. A request can name an adapter revision long after its bytes have left the local engine. The request first resolves a user-facing policy name to an exported revision and an engine-local adapter id. If the adapter is already active in the GPU batch, decoding can use it immediately. If it is cached in CPU memory, the engine promotes it. If it is absent from the local cache, the serving actor fetches the adapter file from shared storage and loads it before decoding. This lets MinT keep a large addressable catalog while bounding the much smaller CPU-cache and same-batch working sets.

Training has the opposite problem: one resident base deployment may serve several policy records over time. When a trainer switches from policy A to policy B, MinT must save A's adapter, optimizer moments, scheduler position, accumulated gradients, and unconsumed rollout records before restoring B's corresponding state. The base weights stay resident. Only the LoRA tensors and training state change. Policies may also use different LoRA ranks or target-module sets, so the policy record carries the adapter shape that the worker must allocate and restore.

Sparse models split rollout metadata into two cases. MoE rollout records can carry selected expert ids. When the training backend can map those ids to its expert-parallel layout, log-probability scoring reuses the recorded expert path; when the ids are missing or unmappable, MinT removes that token from the replayed policy-gradient term. GLM-5-style dynamic sparse attention uses a different treatment. MinT currently lacks replay for every DSA indexer selection. The rollout record stores the backend/model path and correction policy, not per-token DSA indexer selections, and IcePop-style rollout correction zeroes importance weights whose training/rollout ratio falls outside the trusted band. That mitigation keeps unstable tokens out of the gradient term; it does not reconstruct the exact sparse-attention token set selected by the inference engine.

The adapter revision is the behavior-carrying payload; the policy record is the service state that makes that payload schedulable and durable. In this paper, a deployable LoRA policy over a resident base is the LLM behavior that MinT trains, evaluates, serves, and rolls back. MinT keeps the base deployment, training checkpoint, rollout record, exported adapter

revision, and adapter cache state as separate facts so one trained behavior can be resumed, scored, exported, evicted, reloaded, and served in a larger population.

3 System Design

MinT is a Tinker-compatible managed service for LoRA RL over resident base-model deployments. A client package calls the service to sample rollouts, compute gradients, apply an optimizer step, export an adapter revision, evaluate or serve that revision, and poll for the result. The service keeps the user-facing loop simple while dense and MoE bases remain loaded in resident trainers, samplers, and serving actors. Durable operation ids and policy records keep retries precise: the service knows which adapter generated a rollout, which checkpoint can resume training, which export is visible to samplers, and which resident worker can run the next request.

Figure 1 gives the service view, and figure 2 shows the training-serving boundary. The runtime must preserve both: a simple client interface and an adapter-only handoff. MinT therefore separates service control from resident compute. The service plane validates each call, records a pollable operation id, resolves the policy record or exported revision, and admits work onto a compatible worker. The compute plane has three service roles because different operations on one policy record require different compatible worker shapes. Single-worker PEFT trainers run LoRA updates when one worker can hold the base replica. Distributed Megatron trainer groups run LoRA updates when tensor, pipeline, or expert parallelism partitions the base and adapter tensors across ranks. vLLM sampler and serving actors hold inference bases and attach exported LoRA adapters for rollout, evaluation, and online serving. The runtime path has four mechanisms: service-plane visibility and policy-record resolution, time-sliced training over resident bases, export from trainer state to serving adapter files, and shared-base rollout or serving with adapter cache tiers.

3.1 Service Plane and Resident Workers

The service plane owns the client-visible state for each operation. It decides when a request becomes scheduled work, when a produced file becomes a usable checkpoint, exported revision, or result, and what a retry observes after a worker exits.

Operation visibility.The service plane turns each client call into scheduled work on a compatible resident worker. It validates the request, enqueues the operation, returns an id that the client can poll, and admits the work only when a worker with the required base deployment and adapter capacity is available. A worker may keep temporary tensors while it executes the call. A checkpoint, exported adapter revision, rollout record, or operation result becomes visible only when MinT writes the metadata entry that names its completed files and stores the pollable result. If a worker crashes after writing adapter files but before recording that entry, later requests cannot select those uncommitted files; the caller

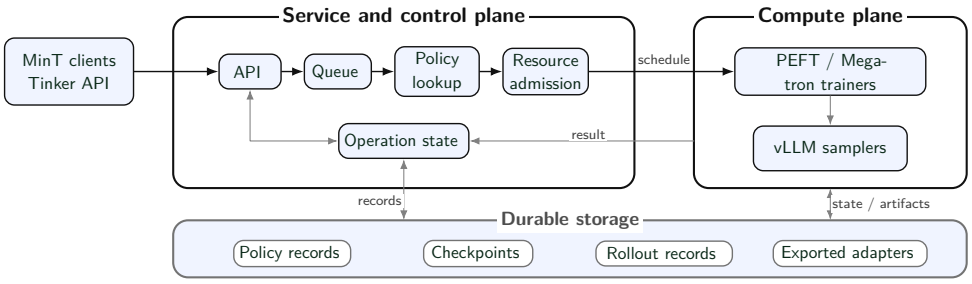


FIGURE 4 MinT runtime organization. Tinker-compatible client packages call the service and receive a pollable operation id. The service queues the operation, reads the policy record, admits work onto the compute plane, and records completed operation results until clients poll them. Trainer workers update LoRA tensors and optimizer state over a resident base model; vLLM sampler and serving actors generate with exported adapter revisions. Durable storage holds checkpoints, rollout records, and exported revisions.

can retry the operation, and cleanup can remove unreferenced attempt paths. The same visibility rule covers training checkpoints, exported adapter revisions, rollout records, and pollable operation results.

Policy record resolution.Every training or sampling call resolves to a policy record before it reaches a worker. The record names the compatible base version, LoRA rank, target modules, and storage locations for checkpoints, rollout records, optimizer state, and exported revisions. Training restores the state named by the record: adapter tensors, optimizer moments, scheduler position, gradients, and rollout metadata. Sampling selects an exported adapter revision from the same record. If the service process restarts, completed checkpoints, exported revisions, rollout records, and finished operation results remain recoverable from storage.

Worker admission and eviction.Training and sampling workers consume cluster GPUs in different shapes, so the service admits them through one resource view. A single-worker PEFT trainer occupies one model replica. A Megatron training group spans tensor-parallel, pipeline-parallel, or expert-parallel ranks. A vLLM sampler reserves memory for a base model plus adapter slots. MinT tracks live workers, active training sessions, in-flight generation, pinned adapters, idle time, and reclaimable base deployments. Evicting an idle trainer frees compute while stored LoRA tensors, optimizer state, rollout records, and exported revisions remain requestable. Evicting an idle sampler removes actor-local cached adapters and GPU-batch slots while the exported revisions remain in shared storage for later loading.

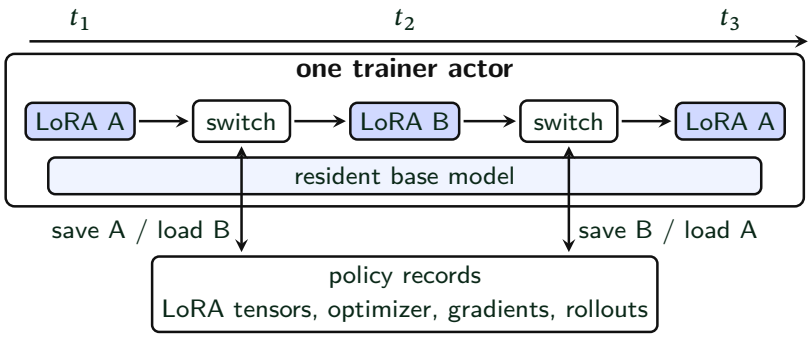


FIGURE 5 Time-sliced multi-LoRA training. One trainer keeps the base model resident. A scheduled policy loads its LoRA tensors, optimizer state, gradients, and rollout records into the trainer; the previous policy writes the same state back to storage before the switch.

3.2 Time-Sliced Multi-LoRA Training

MinT trains many LoRA policies over one resident base without allocating one base replica per policy. Replicating the frozen base for every active policy would spend most GPU memory on repeated weights and would remove the memory advantage that made LoRA attractive. Multi-LoRA training kernels, as explored by mLoRA for fine-tuning workloads [52], can update several adapters in one batch. MinT instead time-slices policies on each trainer because RL requests usually contain enough rollout tokens to form efficient per-policy batches. The trainer swaps only the selected policy's LoRA tensors and optimizer state between requests while the frozen base remains loaded.

Each trainer time-slices locally. The service can run several resident trainers and samplers for the same base family, and the queue assigns ready policy operations across that pool. Each trainer executes one policy training session at a time so optimizer state and accumulated gradients have one writer, while other policies can progress on other workers or in sampler actors.

When policy A yields the trainer to policy B, MinT writes A's training state out and restores B's state before the next gradient computation. The swapped state includes LoRA tensors, optimizer moments, scheduler position, accumulated gradients, and rollout records. After the update, MinT writes the changed state back. The base model stays in GPU memory across policy switches, while inactive adapter tensors and optimizer state can sit in CPU memory or storage until another request selects them.

The service supports different LoRA ranks and target-module sets on the same resident base when the worker was configured for those shapes. Two policies may use different ranks or target different modules, such as attention-only adapters versus adapters on attention, MLP, and output layers. A worker advertises supported adapter-shape limits when it joins the service. Within those limits, the trainer allocates adapter slots at configured

maximum shapes, pads smaller adapters into those slots, and masks inactive rows or modules so each policy updates only its own parameters. Requests outside those limits require a worker configured for the larger rank or target-module set.

Single-worker PEFT trainers and distributed Megatron trainer groups restore LoRA tensors, run the update, and checkpoint the result. For smaller dense models, a trainer restores one adapter into one complete model replica. For Megatron MoE models, a trainer group restores tensor-parallel slices and expert-parallel adapter tensors onto the ranks that own the corresponding base shards. The model-parallel base remains distributed and resident; only LoRA tensors and optimizer state change when the scheduler selects a different policy.

3.3 Adapter Data Flow Between Training and Serving

After the trainer updates a LoRA, the next rollout, evaluation run, or serving request must use that adapter on an inference engine. The trainer holds LoRA tensors, optimizer state, and, for distributed training, rank-local tensor shards. vLLM expects a fixed adapter revision in the serving tensor layout, with optimizer state and rank-local training files removed. MinT exports the trained LoRA in PEFT format, carrying adapter tensors, rank, target modules, and base-model compatibility metadata without copying base checkpoint bytes.

Distributed export rebuilds serving adapter from sharded trainer files. Tensor-parallel ranks may hold slices of the same adapter tensor, and expert-parallel ranks may own different expert adapters. MinT gathers tensor-parallel slices, writes replicated tensors once, collects expert tensors from the ranks that own experts, and emits the PEFT layout expected by the sampler. For MoE adapters with shared-expert LoRA on each expert-parallel shard, the export path deduplicates shared-expert tensors, and the loader materializes one copy instead of repeating it for every shard. The file that crosses from training to rollout or serving is one adapter revision assembled in serving layout; it excludes merged base weights and rank-local training files.

The sampler admits an exported adapter only when its base model family, target modules, rank, and tensor layout match the resident base deployment and configured adapter buffers. The policy may continue training after export, while evaluation and serving select the fixed revision produced by a particular export.

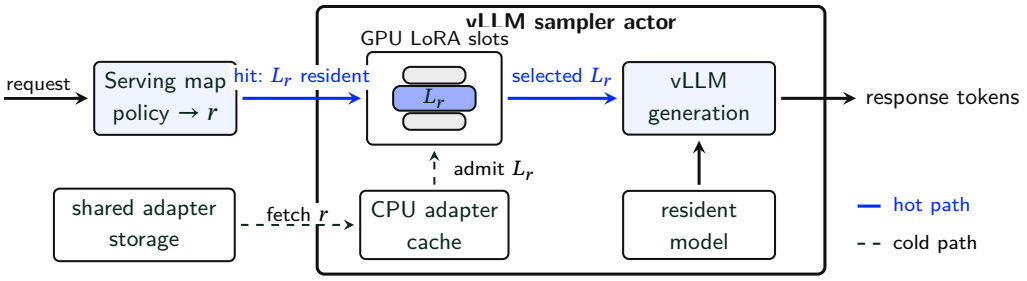


FIGURE 6 Shared-base multi-LoRA sampling. A request reaches the serving map, which resolves the selected policy to exported adapter revision r. The solid path uses revision r because its LoRA L_r is already in a GPU batch slot. The vLLM actor runs inference with the resident base and L_r, then returns response tokens. Dashed arrows show the cold path: fetch revision r from shared storage into the per-sampler CPU cache, then admit L_r into a GPU batch slot before inference.

3.4 Shared-Base Rollout and Serving

MinT uses vLLM for rollout and serving because vLLM can run inference from a resident base with a LoRA adapter resident in a GPU batch slot. vLLM handles token generation. MinT handles revision selection, adapter load queues, adapter cache tiers, and the export path that turns trainer state into serving adapter files. A sampler actor keeps one base deployment resident, admits exported adapters into vLLM's LoRA slots, and runs inference with the adapter named by MinT. The service maps a user-facing policy name to a policy record, chooses an exported adapter revision, and sends the sampler that revision. Figure 6 shows the serving path for a request that resolves to revision r.

Each sampler manages adapter cache state across three tiers, which section 5.3 measures in detail. The addressable catalog holds exported revisions in shared storage. The CPU cache holds adapter bytes already near one sampler actor. The GPU batch holds adapters resident in current inference slots. A request uses a GPU-batch adapter if it is already resident in a slot, promotes a CPU-cached adapter if it is warm, or schedules a cold load from shared storage if the adapter is only in the catalog. Cold adapter loads are scheduled service work before inference: MinT resolves the revision, queues the load job, controls which cold loads enter the sampler, and delays inference starts when too many distinct cold adapters arrive together.

4 Three Scaling Axes

Section 3 defined the training-to-serving path: a rollout is generated by a resident base plus an adapter revision, training updates that adapter, and export returns a PEFT adapter file in serving layout. Scaling MinT stresses three parts of that path: large-base placement, exported adapter bytes, and adapter cache state. Scale up keeps LoRA RL usable when

dense or MoE bases require model-parallel placement and sparse-routing metadata. Scale down makes the adapter revision, rather than a merged checkpoint, the object that crosses from training to serving. Scale out keeps a large addressable policy catalog selectable while each engine admits only bounded CPU-resident and GPU-active working sets.

4.1 Scale Up: LoRA RL on Large Dense and MoE Bases

Megatron placement.MinT uses Megatron training groups when the base model is too large for a single PEFT worker. Tensor parallelism shards dense tensors. Expert parallelism assigns MoE experts to rank groups. Dense-module LoRA tensors follow the tensor-parallel shards for the dense weights they modify. Per-expert LoRA tensors are keyed by expert id: the EP shard owns the experts assigned to it, and TP shards the expert matrices inside that owner group when the run also uses tensor parallelism. Shared-expert LoRA is stored once per EP shard and deduplicated during export.

Policy switching.The base shards stay resident across policies. A policy switch restores the adapter tensors, optimizer moments, and accumulated gradients on the ranks that will execute the next update. This keeps the policy training state small relative to the resident base while preserving Megatron's sharding rules.

Distributed export.Serving needs one PEFT adapter revision that can attach to a resident vLLM base deployment. Export converts the Megatron training view into that serving view: tensor-parallel LoRA slices are gathered, replicated tensors are deduplicated, and expert LoRA tensors are collected from their expert-parallel owners. The exported adapter is the policy revision that later appears in serving, evaluation, rollback, and catalog records.

MoE router replay.MoE RL requires training-time scoring to use the expert path that generated each rollout token. R3 identifies router mismatch as a concrete source of instability in MoE RL: small implementation or precision differences can route the same token to different experts during rollout and training [6, 25]. R3 records MoE expert routing. For Qwen3-style MoE runs, MinT stores selected expert ids with rollout records. Training can replay a recorded route when the backend can reconstruct that expert path. Training masks a token when the rollout record lacks selected expert ids or the training backend cannot map those ids to its expert-parallel layout.

Sparse-attention provenance.Dynamic sparse attention has a separate mismatch channel. In GLM-5 and GLM-5.1, the DSA indexer and top-k path decide which tokens participate in sparse attention; small numerical differences can change that token set [7, 14]. MinT removes observed implementation mismatches where the stack exposes a concrete cause: indexer RoPE layout, normalized query/key inputs, deterministic top-k behavior, frozen indexer defaults, long-context THD/CP support, and LoRA loading for DSA target modules. Probability mismatch can remain after those fixes, so MinT uses IcePop-style rollout correction [21]: when the training/rollout probability ratio leaves the configured lower–upper trusted band, the token receives zero importance weight. This mitigation filters

unsafe scoring terms. It does not replay every DSA indexer choice, and it does not prove that training used the exact sparse-attention token set selected by the inference engine.

Family	Model structure	Scale validated	MinT support path
Qwen3 series	Dense and MoE variants	0.6B/4B dense adapters; 30B-A3B and 235B-A22B MoE runs	Single-worker PEFT, Megatron MoE training, expert-route records, adapter export, and shared-base serving [6, 35]
Moonlight & Kimi K2	MLA MoE	Moonlight-16B-A3B bring-up; Kimi K2 1.04T countdown-task LoRA RL	MLA/MoE adapter placement, Megatron-Bridge conversion, vLLM serving, and trillion-parameter-class LoRA RL [6, 23, 29]
GLM-5 / GLM-5.1	MLA, DSA, MTP, and MoE	Frontier agentic family with DSA/MTP training and serving bring-up	DSA/MTP training patches, DSA LoRA target mapping, vLLM custom-forward LoRA loading, bridge conversion, and IcePop rollout correction [7, 14, 21]

TABLE 1 Representative model-family support validated by the current MinT stack.

4.2 Scale Down: Adapter-Only Training-Serving Handoff

Adapter handoff bytes.The training-serving handoff uses the LoRA adapter as the checkpoint. A measured Qwen3-4B rank-32 PEFT adapter file is 264,310,274 bytes, about 252 MiB. Adapter byte size is determined by rank, target modules, dtype, and tensor layout. A four-billion-parameter bf16 base checkpoint has an 8.0 GB weight floor before metadata and optimizer state. The measured adapter is about 3.3% of that base-weight floor, and the serving path can load it without materializing another full base copy. Since LoRA tensor bytes scale approximately linearly with rank for a fixed target-module set, the same target pattern at rank 1 would be about 7.9 MiB, or roughly 0.10% of the bf16 base-weight floor. This supports the abstract's conservative 1% claim: compact rank-1 settings can fall below 1%, while broader target-module choices and metadata can increase the footprint. The system property remains the same: the crossing artifact is an adapter revision, not a full checkpoint.

Serving-compatible export.MoE export applies the same adapter-revision rule to sharded training runs. The trainer writes a PEFT adapter file in the tensor layout expected by vLLM, with optimizer state and rank-local training files removed. The file that crosses from training to serving is the adapter revision.

Served-score attribution.A served score names the exported adapter revision together with its compatible base model, target modules, prompt renderer, scorer, and serving path. When a row reports a served score, MinT compares isolated adapter loading against shared-base serving before attributing the served score to the exported revision. This check separates the policy result from loader and routing differences.

4.3 Scale Out: Policy-Population Serving

From stored adapters to live policies.Scale-out begins when exported adapters stop being a small set of training files and become a large set of deployable policies. The catalog can grow with tenants, product variants, rollback points, personalization branches, evaluation snapshots, and research sweeps. A user-facing name resolves to an exported adapter revision, the revision resolves to a durable adapter file, and a serving actor maps that file to a local adapter id when the policy enters that actor. Table 2 names the three tiers that a policy revision can occupy at once; the paragraphs below visit them in turn.

Tier	Scale	Lifetime	Promotion / eviction	Measured in this paper
Addressable catalog	10^3–10^6 entries	Durable (control plane)	Promoted by adapter export; retired manually	Catalog sweep 1k–100k (table 6); 10^6-entry sizing in Appendix table 13
CPU adapter cache	Hundreds per engine	Per actor run	Promoted by router or cache-miss load; LRU under memory pressure	369 / 550 cached on one engine (table 6; Appendix table 10)
GPU batch	≤ 64 distinct adapters	One decoding step	Promoted by the batch scheduler; released at end of step	Same-batch frontier N=64 (table 6; Appendix table 10)

TABLE 2 Three cache tiers separate addressability from local residency and same-batch execution. A policy revision can occupy any subset of these tiers at a given moment; the scales, lifetimes, and control surfaces differ.

Catalog size is the request-name scale.The measured 1k–100k catalog sweep keeps name resolution separate from engine-local cache state. The Qwen3-30B TP=4 experiments resolve request names against catalogs up to 100k entries, keep hundreds of adapters cached near one engine, and run clean same-batch probes through 64 distinct adapters. Appendix table 13 extrapolates the measured single-engine limits to a 10^6-entry accumulated catalog under a fleet-level routing model. These measurements split scale-out into three quantities: how many policy revisions can be named, how many can stay cached near an engine, and how many can execute in the current batch. The 10^6 number is addressability, not simultaneous GPU residency.

Traffic locality drives cache state.Catalog membership and local cache state change on different time scales. Catalog entries live in the control plane. Cache entries are local to a serving actor. The same adapter revision can be addressable in the catalog, cached in CPU memory on one actor, active in the current GPU batch, evicted back to shared storage, and later loaded again. Recurring traffic should stay near an engine that already holds the selected policy, while weak-locality sweeps and rollout waves expose cache misses.

Cold loading is service work.A cache miss does more than read a small LoRA file. The serving actor fetches tensors, materializes loader objects, registers the adapter with the engine, and activates it before decoding can start. Different missing policies serialize through the engine load path, so latency grows as a staircase when many unique policies arrive together. Concurrent cache-miss requests for the same missing policy can share one load; requests for different missing policies remain separate load jobs. MinT therefore treats cold loading as scheduled service work with deduplication and bounded backpressure.

Serving contract.Policy-population serving requires controls after name lookup. Routing preserves CPU-cache reuse when traffic has locality. Batch construction respects the smaller same-batch adapter limit; the addressable catalog is larger than any realized batch. Cold loading is observable, retryable, and backpressured because it turns a stored adapter revision into an engine-local adapter object. The policy record and exported revision stay stable while the adapter moves among shared storage, the CPU cache, and the GPU batch.

Representation controls the load staircase.MoE LoRA makes the cold path visible because a moderate-size adapter can expand into tens of thousands of tiny tensor objects and hundreds of megabytes of cached state across TP workers. MinT packs these tensors into a serving representation with nearly unchanged declared bytes, reducing object fanout

before engine registration. The detailed serving measurements in section 5.3 show 37,248 tensors reduced to 672 and an 8.5–8.7× live-load speedup, with the packed live engine-load slice below 0.2 seconds in the measured bursts. This live-load number is one component of a longer cold path that also includes routing, queueing, fetch, and generation. The exported adapter remains the policy unit, while cold loading becomes an explicit adapter lifecycle stage that can be routed, prewarmed, throttled, and optimized.

5 Evaluation

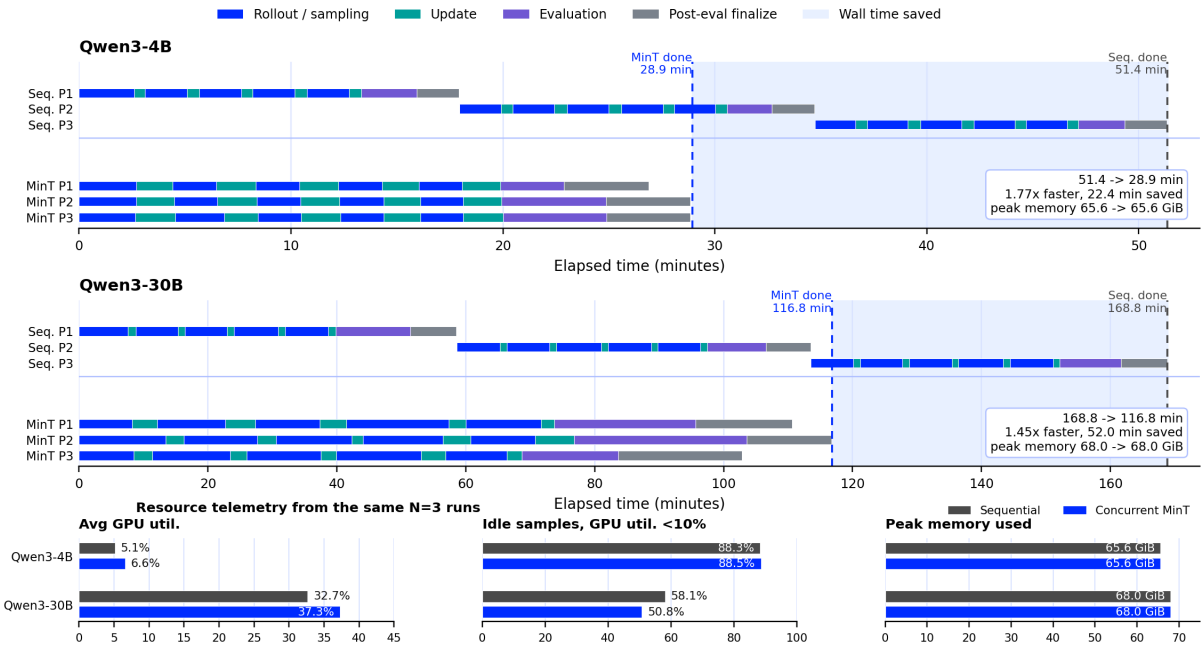


FIGURE 7 Concurrent multi-LoRA training overlaps GRPO runs under the same base-model allocation. Timeline lanes show the schedule, and the lower panels summarize average GPU utilization, samples below 10% utilization, and peak memory from the same runs. Vertical dashed lines mark completion time for the concurrent and sequential schedules.

Model	Schedule	Workloads	Wall time	Time saved	Speedup	Peak memory
Qwen3-4B						
Qwen3-4B	**Sequential**	3 GRPO policies	3081.2 s	–	1.00×	65.6 GiB
Qwen3-4B	**Concurrent MinT**	3 GRPO policies	1736.1 s	1345.1 s / 43.7%	1.77×	65.6 GiB
Qwen3-30B						
Qwen3-30B	**Sequential**	3 GRPO policies	10130.0 s	–	1.00×	68.0 GiB
Qwen3-30B	**Concurrent MinT**	3 GRPO policies	7008.4 s	3121.6 s / 30.8%	1.45×	68.0 GiB

TABLE 3 Concurrent-training schedule summary for three GRPO policies. Time saved and speedup compare each concurrent MinT row against the sequential row for the same model.

The experiments test the three scaling axes through the same LoRA lifecycle. Scale Down is measured by adapter-only handoff against merge-and-load paths and by concurrent training schedules that reuse one resident base allocation. Scale Up is measured by dense SFT, DPO, and GRPO runs, sparse-route MoE RL, Qwen3-235B-A22B GRPO, and a Kimi K2 1T countdown-task path. Scale Out is measured by separating addressable catalog size, CPU-cache size, same-batch adapter diversity, and cold-load cost; the million-scale claim is a catalog and routing claim, not a claim that one engine keeps every adapter resident or active.

An adapter revision means a fixed exported adapter version selected by rollout, evaluation, serving, or rollback. The term names the behavior selected by a request, separate from the file transfer or load operation that may place that adapter near a sampler.

The handoff and utilization measurements isolate the systems effect of making the adapter the training-serving artifact. The learning measurements confirm that the same lifecycle supports the public cookbook recipes and larger sparse-model deployments. The serving measurements show that a large policy catalog is an addressability scale, while CPU-cache residency, same-batch adapter diversity, and cold-load work are the online execution scales. This distinction is the evidence boundary for the abstract: MinT can manage a million-scale catalog while training and serving selected revisions through bounded resident working sets.

SFT denotes supervised fine-tuning. DPO denotes pairwise preference optimization. GRPO denotes rollout-based reinforcement learning. The public MinT cookbook is the recipe layer around the framework: it packages task configurations, benchmark manifests, proxy screens, full confirmations, and maintained adapter recipes [27]. DAPO-AIME24 is the math-RL cookbook recipe evaluated on AIME 2024, chat-DPO is the pairwise-preference recipe, LawBench is the legal-reasoning recipe, and Fineval is a finance-domain supervised benchmark.

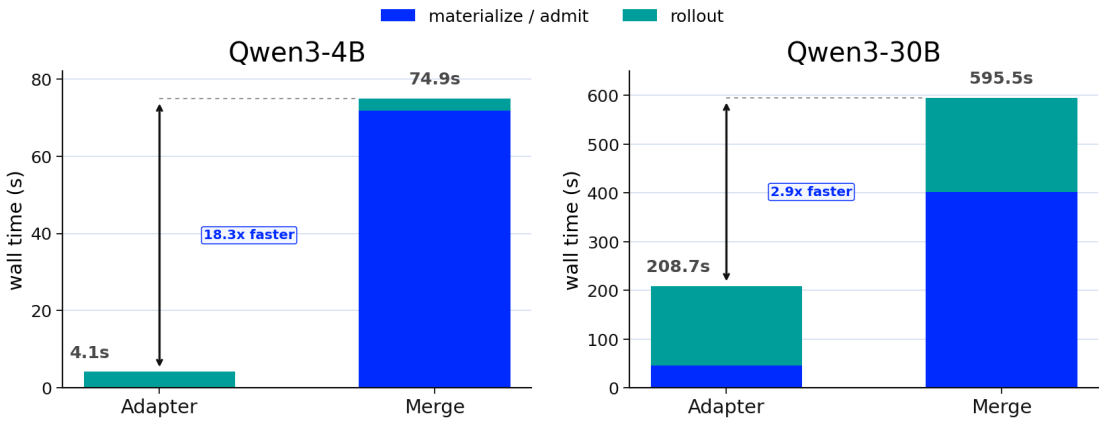


FIGURE 8 Adapter handoff avoids the merge-and-load stages that dominate the full-checkpoint path. Each stacked bar separates materialization or adapter loading from rollout under the probe protocol for the corresponding model.

5.1 Scale Down and Multi-Train Utilization

Adapter handoff compares two ways to send a newly trained policy to the sampler. The merge path materializes a full checkpoint and then loads that checkpoint before rollout. The MinT path loads the exported adapter into a resident shared-base sampler. Figure 8 plots total step time as materialization or adapter loading plus rollout, and table 4 records the file sizes, cold first-sample latency, and total versus warm generation rates used to interpret the bars. The total rate includes the first request in the probe sequence, while the warm rate excludes it. A merged checkpoint may achieve higher or lower token throughput than a LoRA-based adapter during rollout, so the end-to-end comparison involves a trade-off between the cost of shipping the artifact and the resulting sampling throughput. In these runs, loading the adapter saves enough materialization and loading time to dominate the rollout-speed differences.

Model	Path	Checkpoint parameters	Checkpoint file size	Materialization or load	Cold first sample	sample speed total/warm
Qwen3-4B						
Qwen3-4B	**Adapter**	rank-32 LoRA	252 MiB	0.036 s	4.114 s	15.568/15.567 tok/s
Qwen3-4B	**Merge**	full model	8.061 GB	71.820 s	55.704 s	4.697/20.595 tok/s
Qwen3-30B						
Qwen3-30B	**Adapter**	rank-16 LoRA	1.692 GB	46.455 s	117.304 s	1.874/5.700 tok/s
Qwen3-30B	**Merge**	full model	61.084 GB	402.245 s	156.074 s	1.573/6.904 tok/s

TABLE 4 File and sampling metrics for the adapter-handoff comparison in figure 8. Cold first sample is the first request wall time. The total sample speed includes that first request, while the warm sample speed excludes it.

Concurrent training measures schedule utilization under one resident base allocation. A

sequential schedule can fit in memory while leaving the resident base idle between rollout, update, export, and evaluation phases. MinT keeps the same peak memory budget and lets other policies use those idle periods on resident trainers and samplers. The measured speedup comes from filling cross-policy gaps in the schedule, while each rollout and optimizer step still runs the same model computation for its selected policy. In the completed GRPO runs, concurrent execution finishes in 1736.1 s instead of 3081.2 s on Qwen3-4B and in 7008.4 s instead of 10130.0 s on Qwen3-30B, while peak memory remains unchanged within each model size. Figure 7 shows the measured schedule timeline and matching GPU telemetry, and table 3 records the schedule summaries.

The timing and utilization experiments isolate the systems effect of the adapter design. Learning quality is evaluated under the same adapter lifecycle in the next group of experiments.

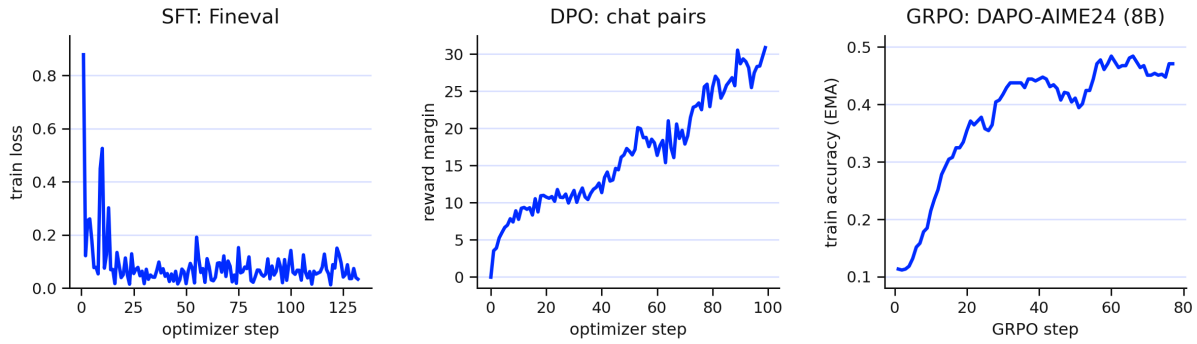


FIGURE 9 Dense-model learning traces. Each panel keeps the native metric for its training paradigm instead of forcing SFT loss, DPO reward margin, and GRPO train accuracy onto one axis.

5.2 Scale Up Across Training Paradigms and Model Scales

Dense-model experiments use Qwen3-4B for SFT and DPO, and Qwen3-8B base for GRPO. The SFT rows use FinEval and finance-sentiment benchmarks from the FinGPT evaluation suite [15, 24]. The DPO row uses the chat-DPO recipe's reward-margin trace. The GRPO row uses the Qwen3-8B-base DAPO-AIME24 trace on the 2024 American Invitational Mathematics Examination [26]. Figure 9 shows the learning traces, while table 5 separates curve-backed rows from held-out confirmation rows.

Each paradigm exercises a different part of the same lifecycle. SFT tests whether a supervised dataset moves into the adapter, trains through the LoRA path, and produces held-out gains comparable to a full fine-tune; the five FinGPT-suite rows cover finance-domain accuracy and the broader FinEval task at the same time. DPO tests whether a preference-pair objective drives the same adapter through the chat-DPO recipe and increases the chosen-minus-rejected reward margin during optimization. GRPO tests whether a rollout-based RL recipe updates the adapter from on-policy AIME 2024 rollouts. Together, the rows show one adapter type and one export format carrying SFT loss, DPO reward margin,

and GRPO train accuracy without any per-paradigm tooling change. Table 5 reports the endpoint rows that quantify each of these checks.

Paradigm	Benchmark	Metric	Evidence	Result
SFT: finance suite				
SFT	Fineval	accuracy	held-out score	0.4226 → 0.7811
SFT	FPB	accuracy	held-out score	0.6906 → 0.8804
SFT	FiQA-SA	accuracy	held-out score	0.8255 → 0.8473
SFT	TFNS	accuracy	held-out score	0.5959 → 0.9095
SFT	NWGI	accuracy	held-out score	0.4954 → 0.5925
Preference optimization				
DPO	chat pairs	reward margin	trace endpoint	−0.03 → 30.88
Reinforcement learning				
GRPO	AIME24	train accuracy (EMA)	Qwen3-8B trace	0.11 → 0.47; best raw 0.568 at step 76

TABLE 5 Dense-model result rows. The table keeps the native metric for each objective and separates plotted traces from held-out confirmation scores. FPB, FiQA-SA, TFNS, and NWGI are finance-sentiment held-out sets from the FinGPT evaluation suite [24].

The dense rows separate two kinds of evidence under the same adapter lifecycle. The five SFT rows from the FinGPT evaluation suite all confirm large held-out gains over the base Qwen3-4B score: roughly +36 points on FinEval, +19 on FPB, +31 on TFNS, and smaller but consistent moves on FiQA-SA and NWGI. The DPO row reports the reward-margin endpoint from the chat-DPO optimization trace. The GRPO row reports the Qwen3-8B-base DAPO-AIME24 training trace, whose EMA train accuracy rises from 0.11 to 0.47 and whose raw best point reaches 0.568 at step 76. The combined message is paradigm-agnostic: the same lifecycle carries supervised, preference-based, and rollout-based updates without any per-paradigm checkpoint surgery.

MoE experiments add two conditions beyond the dense rows. First, expert routes must be replayed during training-time scoring so that tokens are scored under the MoE path that generated them. Second, large MoE models test whether the adapter/base split survives distributed placement across tensor-parallel and expert-parallel workers. The Qwen3-235B-A22B run uses the Hopper-class `mint-prod-aliyun` profile: a 32-GPU Megatron trainer with TP=4 and EP=8 (PP=1), paired with a 16-GPU TP=16 serving deployment. The Kimi K2 1.04T countdown-task run uses a 64-GPU H800 deployment with the same LoRA RL path on 32.6B active parameters. Figure 10 shows the 30B and 235B AIME24 curves together with the Kimi K2 1T countdown-task RL curve [23].

FIGURE 10 MoE RL curves. The 30B/235B panels use AIME24 mean@1 with aligned y axes and smoothed overlays. The Kimi K2 panel gives the end-to-end LoRA RL reward curve for the 1T countdown-task run [23].

The 30B-A3B and 235B-A22B panels share the same y axis to make the scale jump readable. The 30B-A3B AIME24 curve rises from a noisy near-zero start to a stable mid-band by the end of the logged window, while the 235B-A22B curve reaches 0.967 peak mean@1 – close to saturation on AIME24 under the same LoRA RL path. The Kimi K2 panel switches to task reward instead of an AIME-style accuracy because the countdown task has a different correctness target, but the curve follows the same rollout-update-export-evaluate loop on a 1.04T-parameter base. Together, the three panels close the scale-up claim along the adapter lifecycle: the LoRA adapter remains the policy object across a 30B sparse base, a 235B-A22B Hopper deployment, and a trillion-parameter MoE, with no change to the training-serving handoff between them.

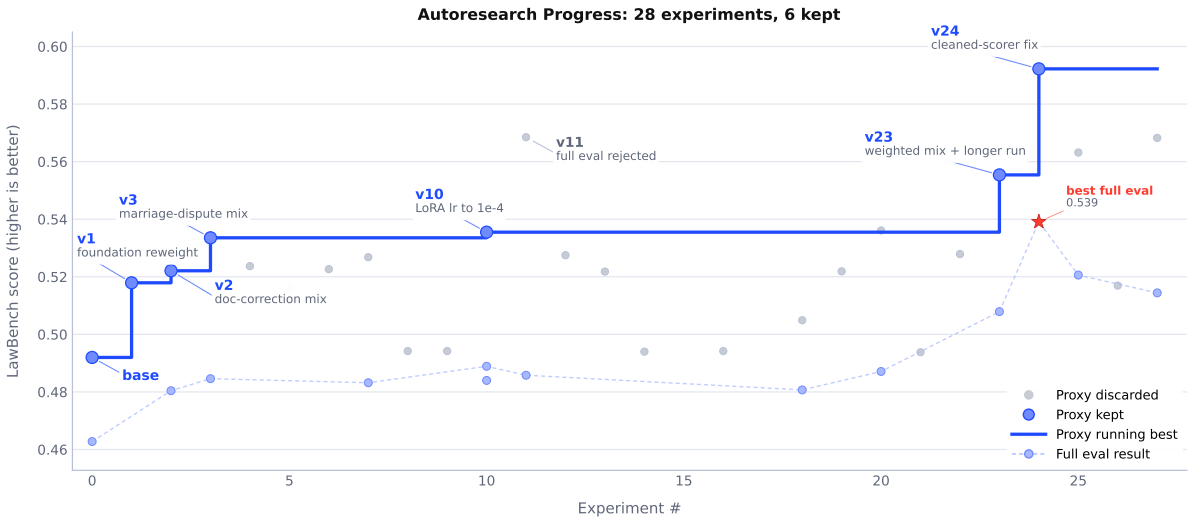


FIGURE 11 LawBench AutoResearch trace from the cookbook utilities. Pale gray points are proxy-screened candidates that were not promoted, blue-outlined points are kept proxy candidates, the blue step line is the running best among kept candidates, violet points are full LawBench evaluations, and the black diamond is the full-manifest control. The labeled v11 proxy high was rejected after full-benchmark confirmation.

The MoE route counters report how often token-level scoring attempts needed route-consistency handling. On Qwen3-30B, the run with R3 has a mean out-of-route scoring ratio of 0.0013% across 87 logged steps, while the comparable no-R3 run averages 0.0097% across 50 logged steps. On Qwen3-235B, the R3 run averages 0.0253% across 88 logged steps. These counters are small because they are token-level rates, yet they identify the exact failure channel that R3 controls: a token sampled under one expert route should not be scored as if a different route had produced it. The Qwen3-235B curve reaches 0.967 peak mean@1 on AIME24.

AutoResearch evaluates recipe candidates in two stages. Proxy LawBench tasks screen many data-manifest and recipe variants, and selected variants then receive full LawBench confirmation before being counted as improved recipes [11]. FinGPT, DAPO-AIME, and chat-DPO supply confirmed SFT, GRPO, and DPO rows above. The cookbook provides AutoResearch utilities for coding agents to launch MinT runs, screen candidate recipes on proxy LawBench tasks, and promote selected candidates to full LawBench confirmation. In the LawBench run, the base Qwen3-4B full score is 0.4628. The v10 recipe is the earlier learning-rate-tuned line. The high-proxy v11 candidate remains discarded because its full LawBench score is 0.4858, below v10's 0.4889. The full-manifest control reruns the later data manifest without the promoted changes and reaches 0.4712. The maintained v23 weighted-aligned recipe extends the aligned-data line to a fairer exposure budget; it reaches 0.5554 on the proxy screen at step 100 and 0.5079 on the full benchmark [27]. Figure 11 plots the proxy-search trajectory and full-benchmark confirmation points.

The proxy-versus-full divergence is the methodological signal the cookbook is built around. The v11 candidate set a new proxy high but its full LawBench score fell below v10's; this is the exact failure mode an unsupervised proxy can produce when its task distribution diverges from the full benchmark's. The full-manifest control at 0.4712 shows that the later data manifest alone does not produce the gain, which separates data effects from recipe effects in the same trace. The maintained v23 weighted-aligned line is the candidate that survives both stages, with a proxy reading of 0.5554 at step 100 and a full reading of 0.5079, well above the base 0.4628 and the v10/v11 cluster. Reading the running-best line together with the kept-candidate filter is how AutoResearch decides to record progress instead of celebrating a single high point, and it is the same lifecycle pattern the previous paradigms use: train an adapter, freeze it as a revision, evaluate that exact revision, then decide whether it enters the maintained recipe set.

Resource tier	Question	Measured bound	Takeaway
Control-plane addressability			
Addressable catalog	How many policy revisions can requests select from?	Catalog sweep from 1k to 100k adapters	Catalog size stays a name-resolution scale; local cache state determines warm/cold latency.
Local adapter cache			
CPU adapter cache	How many adapters can one actor keep resident?	369 loaded adapters at a 512-adapter hotset; 550 loaded adapters under 2048-adapter weak-locality pressure	CPU memory holds hundreds of adapters; weak locality raises tail latency.
Adapters in one GPU batch	How many adapters can decoding use?	64 distinct adapters in the tested same-batch window	Batch execution has the smallest measured adapter-diversity window.
Cold-load path			
Cold loading	What happens on cache misses?	Warm $N = 64$ p95 21.35 s; cold-cache p95 199.81 s; $N = 16$ cold-load staircase 1.375–23.267 s	Different missing adapters are loaded one at a time before decoding.
Adapter representation	How costly is tensor fanout?	37,248 tensors to 672 tensors; 1.05× byte change; 8.5–8.7× faster live load	Packing speeds the cache-miss load path by removing small-object fanout.

TABLE 6 Policy-population serving bounds on one Qwen3-30B rank-1 serving actor with TP=4.

5.3 Policy-Population Serving

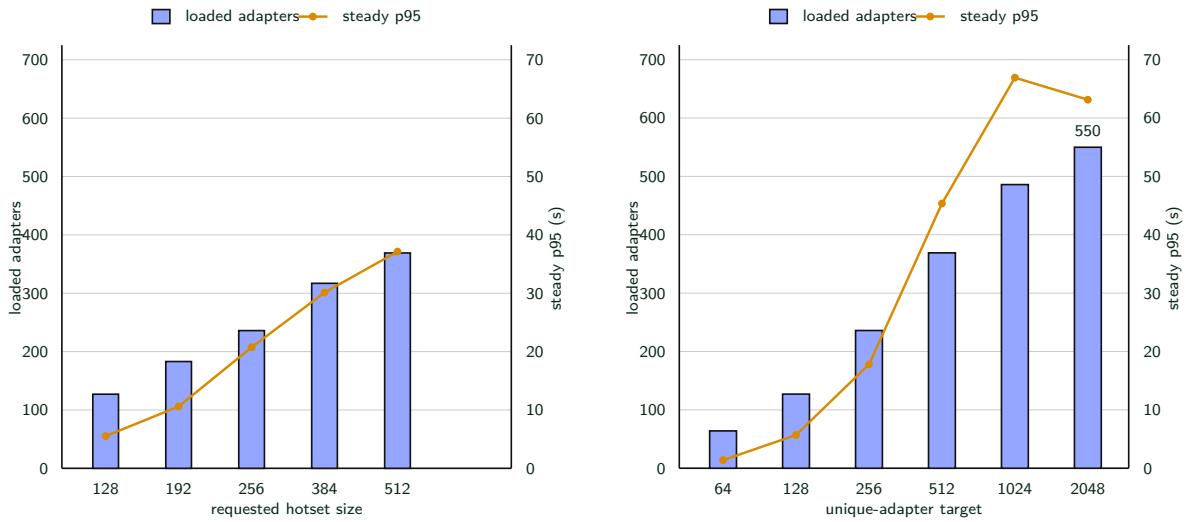
Multi-LoRA serving becomes a policy-population problem when many exported adapter revisions are deployable at once. Training still exports a LoRA adapter instead of a full checkpoint, and serving still keeps the base model resident. A request names one policy revision; the serving actor either finds the adapter in the GPU batch, promotes it from the CPU cache, or loads it from shared storage after a cache miss. Only the policies selected by the current batch consume GPU-batch adapter slots.

The serving experiments use Qwen3-30B rank-1 MoE LoRA adapters on one 4-GPU tensor-parallel serving actor, with prompt length 1024 and maximum output length 64. The probes cover the path from request name to generation: catalog expansion from 1k to 100k entries, repeated-hotset and unique-adapter traffic, warm and cold latency, the tested same-batch adapter window, CPU-cache growth, cold-load accounting, and packed tensor loading. Appendix table 13 extends these single-engine measurements into a 10^6-entry addressable-catalog sizing sketch; the main measurements do not imply that 10^6 adapters are simultaneously resident or active. Table 6 gives the main bounds.

Adapter cache levels.MinT treats each exported adapter revision as an addressable policy revision that can move through three cache levels. The durable catalog names policy revisions that can be requested; the CPU cache is local to one serving actor; the GPU batch is the smaller set attached to a decoding step. A cold load moves a policy revision from the durable catalog into the actor before decoding can begin.

Large catalogs split fleet-level routing from engine-local memory. A million accumulated policy revisions is a catalog and routing problem; one engine keeps only a bounded local cache. The main experiments measure warm/cold behavior across independently run catalogs up to 100k adapters, while Appendix table 13 turns the measured single-engine limits into a 10^6-adapter capacity model under a 2300-distinct-adapter active-wave assumption. Appendix table 8 gives the byte and memory accounting behind these cache levels: HBM remains dominated by the resident base model, while CPU memory and adapter object shape determine how many policies can be kept cached near one engine.

Cached adapters and batch diversity.Local adapter caches absorb locality before requests touch shared storage. Tenant variants, rollback points, personalization branches, and recent evaluation candidates often recur; broad rollout waves and experiment sweeps have much weaker locality. The service has to support both traffic shapes while keeping CPU cache size separate from same-batch execution.



(a) Repeated-adapter traffic.

(b) Unique-adapter traffic.

FIGURE 12 Cached-adapter scaling on one 4-GPU Qwen3-30B rank-1 serving actor. Bars use the left axis for loaded adapters; lines use the right axis for steady p95 latency. The left panel measures locality absorption under repeated-adapter traffic, reaching 369 loaded adapters at a 512-adapter working set. The right panel measures cache growth under weak locality, reaching 550 loaded adapters at a 2048-adapter unique target while p95 rises to 63.14 s.

Figure 12 measures these two regimes. Repeated-adapter traffic reaches 369 loaded adapters at a 512-adapter hotset with p95 37.13 s and no errors. Weak-locality traffic reaches 550 loaded adapters at a 2048-adapter unique target with p95 63.14 s and no errors, so it is cache-growth evidence under pressure rather than a clean warm-cache endpoint. Up to target 512 the two panels report essentially the same loaded count (127, 236, and 369), so locality matters only once the request stream exceeds the working set; beyond that, weak-locality p95 saturates rather than diverges, peaking at 66.92 s at target 1024 and

settling at 63.14 s at target 2048 as the CPU cache plateaus near 550. Both loaded-adapter counts are larger than the tested same-batch window of 64 distinct adapters: one actor can cache more adapters than it can execute together in one GPU batch. Routing should preserve warm reuse, while batching still has to respect the smaller same-batch adapter limit. Appendix table 10 gives the ordered ladder data, and Appendix table 11 shows stress probes where long outputs, weak locality, and high concurrency make simple warm-cache assumptions break down.

Cold loading as service work.Cold misses expose the cache-miss work inside a policy population. New adapters, rollback points, long-tail personalization branches, and evaluation snapshots keep producing cache misses. A cold miss fetches adapter tensors, builds loader objects, registers the adapter with the serving engine, and only then starts generation. MinT therefore treats cold loading as scheduled service work, with deduplication for repeated cache-miss requests to the same policy and bounded backpressure when too many distinct missing policies arrive together.

Figure 13 shows why this is a separate capacity dimension. CPU-cached requests take the warm regime. Cache-miss requests take the cold regime. Increasing the independently run catalog from 1k to 100k adapters keeps the same warm/cold latency modes; Appendix table 9 reports the full sweep, including one failed cold request in the 100k row. The right panel isolates the controlled cold-miss component: 16 different cache-miss policies form a load staircase from 1.375 s to 23.267 s, about 1.35–1.40 s per policy. Concurrent cache-miss requests for the same missing policy can share one load, while different missing policies remain separate load jobs. Appendix table 12 decomposes this path into API queueing, shared loads, unique-policy loading, and retryable load rejection.

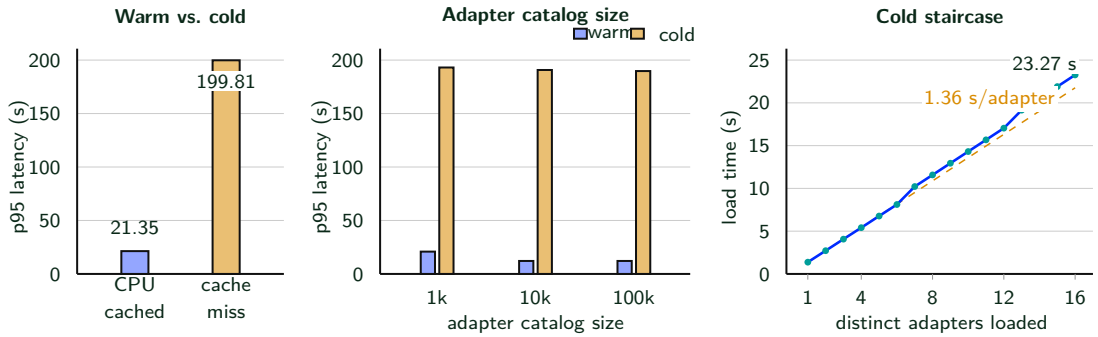


FIGURE 13 Warm routing and bounded cold-load cost. Left: CPU-cached adapters take the warm latency path, while cache-miss policies take the cold path. Middle: warm measurements stay in the warm p95-latency regime and cold measurements stay in the cold p95-latency regime as independently run adapter catalogs grow to 100k entries. Right: 16 different cache-miss policies form a serialized load staircase from 1.375 s to 23.267 s, about 1.35–1.40 s per policy.

Cold-load representation.The load staircase also explains why the adapter file format matters. A rank-1 MoE LoRA adapter is moderate in bytes, while the measured adapter file is fragmented into 37,248 tensor objects, mostly tiny expert tensors. In these probes,

local-disk staging shortened the read path; it left tensor fanout, Python object creation, and loader-side registration work unchanged. MinT therefore packs MoE expert tensors into a serving representation with nearly unchanged declared bytes, reducing object fanout before the engine loads the policy.

Metric	Original	Packed	Effect
Adapter-file shape			
File size	110.75 MB	105.58 MB	1.05× smaller
Tensor objects	37,248	672	55.4× fewer
Cold-load slice			
Read tensors	0.3669 s	0.0067 s	54.8× faster
Build loader objects	0.7540 s	0.0256 s	29.5× faster
Live engine loading			
$N = 4$ **live load**	1.363 s	0.156 s	8.7× faster
$N = 8$ **live load**	1.361 s	0.159 s	8.6× faster
$N = 16$ **live load**	1.388 s	0.164 s	8.5× faster

TABLE 7 Packed MoE LoRA loading reduces cold-load overhead by removing tensor fanout. The byte-size change is small; the speedup comes from replacing many tiny tensor objects with a compact serving representation. The final three rows report the median duration of one live load call measured at each cache-miss burst size.

Table 7 shows the effect. Packing changes file size only from 110.75 MB to 105.58 MB and reduces tensor objects from 37,248 to 672. The direct loader slice improves by 29.5–54.8×, and live engine loading for $N = 4$, $N = 8$, and $N = 16$ improves by 8.5–8.7×, with packed live-load medians under 0.2 seconds. This number refers to the live engine-load slice after packing; end-to-end cold latency still includes routing, queueing, fetch, and generation. The measured speedup comes from object layout: the serving representation removes the small-object storm on cache misses while the declared tensor bytes stay nearly unchanged.

Together, these measurements locate the serving work for policy populations. Catalog lookup and raw LoRA count are control-plane scales in the measured sweep; CPU-cache size, tested same-batch diversity, and unique cold-load cost are online execution scales. Routing and prewarming turn recurring traffic into CPU-cache hits. Batch construction respects the smaller same-batch adapter limit. Cache misses enter an explicit cold-load stage before sampling latency begins. Packing reduces the load staircase itself, making cache misses cheaper without changing the base-model deployment or the addressable policy revision. Appendix section B contains the full ladders, accounting tables, stress probes, and fleet-level sizing sketch behind this contract.

6 Related Work

Post-training workload.MinT targets a workload where post-training leaves many policy variants over a smaller set of base models. Yao argues that future progress shifts more weight toward problem definition, evaluation, and agent-environment interaction [51]. Silver and Sutton frame future agents as learning from streams of experience in addition to static human data [45]. Recent frontier-model reports expose the same systems pressure through reasoning, coding, tool use, executable environments, asynchronous agent RL, and long-horizon evaluation [2, 3, 8, 9, 14, 19, 28, 33, 36]. These workloads produce task variants, evaluation candidates, product branches, tenant adapters, rollback points, and personal adapters that share base models while retaining separate histories.

Service interfaces.Remote post-training interfaces make training loops programmable. Tinker exposes low-level post-training primitives through a remote service [46, 47]. SkyRL tx implements a Tinker-style backend and documents model and benchmark coverage across Qwen3 dense and MoE models, Llama 3, DeepSeek V3, GSM8K, and DAPO/AIME recipes [32]. OpenTinker frames a broader RL-as-a-Service stack around agents, environments, protocols, scheduling, training, and inference [34]. Tinker specifies how users program remote post-training; MinT keeps a compatible programming surface and defines the LoRA-specific service path underneath it: policy record, training checkpoint, rollout record, exported adapter revision, and adapter cache state.

RL execution systems.HybridFlow/verl, AReaL, OpenRLHF, Relax, ROLL, StreamRL, AsyncFlow, Laminar, and NeMo-Aligner study rollout scheduling, colocated and disaggregated execution, asynchronous optimization, GPU utilization, failure isolation, and end-task quality [1, 4, 12, 17, 38, 40, 41, 42, 55]. Their central systems objects are actors, rollout replicas, parameter services, queues, and placement groups. MinT uses the same execution concerns and adds LoRA-specific service state: adapter revisions, optimizer state, rollout records, MoE route records, DSA correction metadata, exported adapters, and adapter cache state stay tied to the policy that produced them.

Training-serving consistency.Modern RL pipelines often generate rollouts through an inference engine and score them through a training backend. Yao et al. study token-probability gaps in hybrid vLLM/FSDP pipelines and propose truncated importance sampling [50]. JetRL shows that mixed BF16 training and FP8 rollout can destabilize on-policy RL under long generations, then uses a unified precision flow to reduce numerical mismatch [49]. R3 studies MoE router disagreement between training and inference and reuses inference-time expert-route records during training [25]. MinT addresses the same problem by storing the adapter revision, rollout record, MoE route or DSA correction metadata, and served evaluation path that name which behavior generated and scored a token.

Parameter-efficient tuning and multi-LoRA training.LoRA freezes the base model and trains low-rank matrices attached to selected layers [16]. AdaLoRA allocates rank budget across matrices according to importance [54]. QLoRA combines frozen quantized bases

with LoRA updates for memory-efficient fine-tuning [10]. LoRA Without Regret argues that LoRA can reach strong post-training quality [39], supporting the premise that LoRA can be more than a memory-saving approximation. MinT turns that premise into managed infrastructure for RL service operation: each update restores one adapter and optimizer state over a resident base, each export produces a serving revision, and each rollout or serving request selects that revision through cache and residency state.

Multi-LoRA serving.Multi-LoRA serving systems optimize inference after adapters already exist. Punica, S-LoRA, dLoRA, dynamic operator optimization, and vLLM improve batching, memory management, scheduling, and kernels for many adapters over a shared base [5, 20, 43, 48, 56]. Compress then Serve reduces adapter storage and serving overhead through individual and joint LoRA compression [13]. FastLibra manages LoRA and KV cache dependencies in a unified HBM cache [53]. LoRAServe handles rank heterogeneity, placement, and routing across distributed LoRA serving clusters [18]. MinT connects the serving catalog to the training lifecycle: an addressable adapter is an exported revision of a trained policy, and a cache miss reloads that revision from the policy's exported adapter file.

Large-model infrastructure.Ray provides the distributed execution framework used by many AI systems [30]. Megatron-LM, Efficient Megatron-LM, ZeRO, and MoE Parallel Folding provide the model-parallel, memory-partitioning, and MoE-parallel techniques that make large-model and sparse-MoE training practical [22, 31, 37, 44]. DeepSeek-V4 describes large-model infrastructure through concrete state units such as hybrid KV cache entries, state caches, on-disk cache storage, rollout WALs, and teacher-state scheduling [9]. MinT applies the same resource-accounting style to LoRA RL: base deployments stay resident, adapter revisions move across subsystems, and adapter cache state controls which exported policies are hot.

7 Conclusion

Post-training now couples rollout, update, evaluation, serving, scheduling, and data movement around many policy variants over a small number of expensive base-model deployments. Full-checkpoint workflows make every task branch, benchmark candidate, product version, tenant variant, rollback point, or personal policy look like another complete model deployment. This abstraction does not scale when the target workload is a large and continuously changing population of trained behaviors over shared frontier bases.

MinT makes exported LoRA adapter revisions the managed unit for this setting. Base-model deployments remain resident, while adapter revisions carry trained behavior through rollout, update, export, evaluation, serving, and rollback. The adapter revision is the behavior-carrying payload; the policy record is the service state that makes that payload reproducible, schedulable, and durable across worker changes, training checkpoints, rollout records, and serving-cache movement.

The measurements validate the same adapter-revision path along three scaling axes. *Scale Up* supports LoRA RL on dense and MoE architectures with model-parallel training and serving paths exercised beyond 1T total parameters. *Scale Down* removes full-checkpoint materialization from the training-serving handoff: adapter-only handoff reduces the measured handoff step by 18.3× on a 4B dense model and by 2.85× on a 30B MoE model, while concurrent multi-policy GRPO shortens wall time by 1.77× and 1.45× under the same resident-base allocation. *Scale Out* separates durable policy addressability from CPU/GPU hot working sets: the serving path supports 10^6-scale addressable policy catalogs, with measured single-engine sweeps through 100K entries and thousand-adapter active waves at cluster scale; first-touch cold loading becomes scheduled service work, and packed MoE LoRA tensors improve live engine loading by 8.5–8.7×.

Together, these results show that multi-tenant LoRA training services can be made practical without turning every trained policy into a full-model server. MinT lets policy count grow through adapter revisions, policy records, cache tiers, and controlled cold-load admission, while selected revisions train and serve over bounded resident working sets. This makes LoRA a service-level unit for large-scale post-training today and a practical path toward larger populations of organizational and personal policies over shared 1T-class base models.

References

[1] Alibaba ROLL Contributors. ROLL: Large-scale reinforcement learning optimization library. https://github.com/alibaba/ROLL, 2025.

[2] Anthropic. Introducing Claude Opus 4.5. Anthropic news, 2025. Accessed 2026-05.

[3] Anthropic. Measuring AI agent autonomy in practice. Anthropic research, 2026. Accessed 2026-05.

[4] AsyncFlow Authors. AsyncFlow: An asynchronous streaming RL framework for efficient LLM post-training. *arXiv preprint arXiv:2507.01663*, 2025.

[5] Lequn Chen, Zihao Ye, Yongji Wu, Danyang Zhuo, Luis Ceze, and Arvind Krishnamurthy. Punica: Multi-tenant LoRA serving. In *Proceedings of Machine Learning and Systems (MLSys)*, 2024. arXiv:2310.18547.

[6] Steven Chiang, Yiwen Lu, Qihan Liu, Andrew Chen, Pony Ma, and Mind Lab. Router replay R3: Why it failed and how we fixed it. Mind Lab: A Lab for Experiential Intelligence, 2026. https://macaron.im/mindlab/research/router-replay-r3-why-it-failed-and-how-we-fixed-it.

[7] Steven Chiang, Yiwen Lu, Qihan Liu, Nolan Ho, Andrew Chen, Pony Ma, and Mind Lab. Support glm5 and glm5.1 in mint: Lora training for dsa and mtp. Mind Lab: A Lab

for Experiential Intelligence, 2026. https://macaron.im/mindlab/research/support-glm5-and-glm51-in-mint-lora-training-for-dsa-and-mtp.

[8] DeepSeek-AI. DeepSeek-V4 release. DeepSeek release notes, 2026. Accessed 2026-05.

[9] DeepSeek-AI. DeepSeek-V4 technical report. *Technical report*, 2026.

[10] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. *Advances in Neural Information Processing Systems*, 2023. arXiv:2305.14314.

[11] Zhiwei Fei, Xiaoyu Shen, Dawei Zhu, Fengzhe Zhou, Zhuo Han, Songyang Zhang, Kai Chen, Zongwen Shen, and Jidong Ge. LawBench: Benchmarking legal knowledge of large language models. *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024. arXiv:2309.16289.

[12] Wei Fu et al. AReaL: A large-scale asynchronous reinforcement learning system for language reasoning. *arXiv preprint arXiv:2505.24298*, 2025.

[13] Rickard Brüel Gabrielsson, Jiacheng Zhu, Onkar Bhardwaj, Leshem Choshen, Kristjan Greenewald, Mikhail Yurochkin, and Justin Solomon. Compress then serve: Serving thousands of LoRA adapters with little overhead. *arXiv preprint arXiv:2407.00066*, 2024.

[14] GLM-5-Team. GLM-5: From vibe coding to agentic engineering. *arXiv preprint arXiv:2602.15763*, 2026.

[15] Xin Guo, Haotian Xia, Zhaowei Liu, Hanyang Cao, Zhi Yang, Zhiqiang Liu, Sizhe Wang, Jinyi Niu, Chuqi Wang, Yanhui Wang, Xiaolong Liang, Xiaoming Huang, Bing Zhu, Zhongyu Wei, Yun Chen, Weining Shen, and Liwen Zhang. FinEval: A chinese financial domain knowledge evaluation benchmark for large language models. *arXiv preprint arXiv:2308.09975*, 2023.

[16] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.

[17] Jian Hu et al. OpenRLHF: An easy-to-use, scalable and high-performance RLHF framework. *arXiv preprint arXiv:2405.11143*, 2024.

[18] Shashwat Jaiswal, Shrikara Arun, Anjaly Parayil, Ankur Mallick, Spyros Mastorakis, Alind Khare, Chloi Alverti, Renee St Amant, Chetan Bansal, Victor Rühle, and Josep Torrellas. Serving heterogeneous LoRA adapters in distributed LLM inference systems. *arXiv preprint arXiv:2511.22880*, 2025.

[19] Kimi Team. Kimi K2.5: Visual agentic intelligence. *arXiv preprint arXiv:2602.02276*, 2026.

[20] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the ACM SIGOPS Symposium on Operating Systems Principles (SOSP)*, 2023.

[21] Ling Team, Anqi Shen, Baihui Li, Bin Hu, Bin Jing, Cai Chen, Chao Huang, Chao Zhang, Chaokun Yang, Cheng Lin, et al. Every step evolves: Scaling reinforcement learning for trillion-scale thinking model. *arXiv preprint arXiv:2510.18855*, 2025. doi: 10.48550/arXiv.2510.18855. URL https://arxiv.org/abs/2510.18855. Introduces IcePop token-level discrepancy masking and clipping.

[22] Dennis Liu, Zijie Yan, Xin Yao, Tong Liu, Vijay Korthikanti, Evan Wu, Shiqing Fan, Gao Deng, Hongxiao Bai, Jianbin Chang, Ashwath Aithal, Michael Andersch, Mohammad Shoeybi, Jiajie Yao, Chandler Zhou, David Wu, Xipeng Li, and June Yang. MoE parallel folding: Heterogeneous parallelism mappings for efficient large-scale MoE model training with Megatron core. *arXiv preprint arXiv:2504.14960*, 2026.

[23] Qihan Liu, Steven Chiang, Rio Yang, Alex Yin, Pony Ma, Andrew Chen, and Mind Lab. Building trillion-parameter reasoning rl with 10% gpus. Mind Lab: A Lab for Experiential Intelligence, 2025. https://macaron.im/mindlab/research/building-trillion-parameter-reasoning-rl-with-10-gpus.

[24] Xiao-Yang Liu, Guoxuan Wang, Hongyang Yang, and Daochen Zha. FinGPT: Democratizing internet-scale data for financial large language models. *arXiv preprint arXiv:2307.10485*, 2023.

[25] Wenhan Ma, Hailin Zhang, Liang Zhao, Yifan Song, Yudong Wang, Zhifang Sui, and Fuli Luo. Stabilizing MoE reinforcement learning by aligning training and inference routers. *arXiv preprint arXiv:2510.11370*, 2025.

[26] Mathematical Association of America. 2024 american invitational mathematics examination. https://maa.org/maa-invitational-competitions/, 2024. American Mathematics Competitions.

[27] Mind Lab. MinT Cookbook. GitHub repository: MindLab-Research/mint-cookbook, 2026. Recipe registry and maintained evaluation artifacts.

[28] MiniMax. MiniMax M2.7: Model self-improvement, driving productivity innovation through technological breakthroughs. MiniMax model page, 2026. Accessed 2026-05.

[29] Moonshot AI. Kimi K2: Open agentic intelligence. *arXiv preprint arXiv:2507.20534*, 2025. doi: 10.48550/arXiv.2507.20534. URL https://arxiv.org/abs/2507.20534.

[30] Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elibol, Zongheng Yang, William Paul, Michael I. Jordan, and Ion Stoica. Ray: A distributed framework for emerging AI applications. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2018.

[31] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Korthikanti, Dmitri Vainbrand, Prethvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, Amar Phanishayee, and Matei Zaharia. Efficient large-scale language model training on GPU clusters using Megatron-LM. *arXiv preprint arXiv:2104.04473*, 2021.

[32] NovaSky AI. SkyRL tx: Unifying LLM training and inference. https://pypi.org/project/skyrl-tx/, 2026. Accessed 2026-04.

[33] OpenAI. Introducing GPT-5.5. OpenAI blog, 2026. Accessed 2026-05.

[34] OpenTinker Authors. OpenTinker: A reinforcement learning as a service framework for agentic workflows. *arXiv preprint arXiv:2601.07376*, 2026.

[35] Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025. doi: 10.48550/arXiv.2505.09388. URL https://arxiv.org/abs/2505.09388.

[36] Qwen Team. Qwen3.5. Qwen blog, 2026. Accessed 2026-05.

[37] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. ZeRO: Memory optimizations toward training trillion parameter models. *arXiv preprint arXiv:1910.02054*, 2019.

[38] Relax Authors. Relax: A service-oriented asynchronous reinforcement learning engine for post-training. *arXiv preprint arXiv:2604.11554*, 2026.

[39] John Schulman and Thinking Machines Lab. LoRA without regret. *Thinking Machines Lab: Connectionism*, 2025. doi: 10.64434/tml.20250929. https://thinkingmachines.ai/blog/lora/.

[40] Gerald Shen et al. NeMo-Aligner: Scalable toolkit for efficient model alignment. *arXiv preprint arXiv:2405.01481*, 2024.

[41] Guangming Sheng, Yuxuan Tong, Borui Wan, Wang Zhang, Chaobo Jia, Xibin Wu, Yuqi Wu, Xiang Li, Chi Zhang, Yanghua Peng, Haibin Lin, Xin Liu, and Chuan Wu. Laminar: A scalable asynchronous RL post-training framework. *arXiv preprint arXiv:2510.12633*, 2025.

[42] Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. HybridFlow: A flexible and efficient RLHF framework. In *Proceedings of the Twentieth European Conference on Computer Systems (EuroSys)*, 2025. doi: 10.1145/3689031.3696075. arXiv:2409.19256.

[43] Ying Sheng, Shiyi Cao, Dacheng Li, Coleman Hooper, Nicholas Lee, Shuo Yang, Christopher Chou, Banghua Zhu, Lianmin Zheng, Kurt Keutzer, Joseph E. Gonzalez, and Ion Stoica. S-LoRA: Serving thousands of concurrent LoRA adapters. *arXiv preprint arXiv:2311.03285*, 2023.

[44] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-LM: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.

[45] David Silver and Richard S. Sutton. Welcome to the era of experience. https://storage.googleapis.com/deepmind-media/Era-of-Experience%20/The%20Era%20of%20Experience%20Paper.pdf, 2025. Accessed 2026-05.

[46] Thinking Machines Lab. Announcing Tinker. https://thinkingmachines.ai/blog/announcing-tinker/, 2025. Accessed 2026-04.

[47] Thinking Machines Lab. Tinker Cookbook. GitHub repository, https://github.com/thinking-machines-lab/tinker-cookbook, 2025. Post-training with Tinker. Accessed 2026-04.

[48] Bingyang Wu, Ruidong Zhu, Zili Zhang, Peng Sun, Xuanzhe Liu, and Xin Jin. dLoRA: Dynamically orchestrating requests and adapters for LoRA LLM serving. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024.

[49] Haocheng Xi, Charlie Ruan, Peiyuan Liao, Yujun Lin, Han Cai, Yilong Zhao, Shuo Yang, Kurt Keutzer, Song Han, and Ligeng Zhu. Jet-RL: Enabling on-policy FP8 reinforcement learning with unified training and rollout precision flow. *arXiv preprint arXiv:2601.14243*, 2026.

[50] Feng Yao, Liyuan Liu, Dinghuai Zhang, Chengyu Dong, Jingbo Shang, and Jianfeng Gao. On the rollout-training mismatch in modern RL systems. *NeurIPS 2025 Workshop on Efficient Reasoning*, 2025. URL https://openreview.net/pdf/325f91538e61ba160793adc5029888c00d06fa7a.pdf.

[51] Shunyu Yao. The second half. https://ysymyth.github.io/The-Second-Half/, 2025. Written April 10, 2025. Accessed 2026-05.

[52] Zhengmao Ye, Dengchun Li, Zetao Hu, Tingfeng Lan, Jian Sha, Sicong Zhang, Lei Duan, Jie Zuo, Hui Lu, Yuanchun Zhou, and Mingjie Tang. mLoRA: Fine-tuning LoRA adapters via highly-efficient pipeline parallelism in multiple GPUs. *arXiv preprint arXiv:2312.02515*, 2024.

[53] Hang Zhang, Jiuchen Shi, Yixiao Wang, Quan Chen, Yizhou Shan, and Minyi Guo. Improving the serving performance of multi-LoRA large language models via efficient LoRA and KV cache management. *arXiv preprint arXiv:2505.03756*, 2025.

[54] Qingru Zhang, Minshuo Chen, Alexander Bukharin, Nikos Karampatziakis, Pengcheng He, Yu Cheng, Weizhu Chen, and Tuo Zhao. AdaLoRA: Adaptive budget allocation for parameter-efficient fine-tuning. *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2303.10512.

[55] Yusen Zhong et al. StreamRL: Scalable, heterogeneous, and elastic RL for LLMs with disaggregated stream generation. *arXiv preprint*, 2025.

[56] Changhai Zhou, Yuhua Zhou, Shiyang Zhang, Yibin Wang, and Zekai Liu. Dynamic operator optimization for efficient multi-tenant LoRA model serving. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pages 22910–22918, 2025.

A Author List

Names are listed alphabetically.

Core Contributors.Andrew Chen, Cleon Cheng, Steven Chiang, Nolan Ho, Andrew Lei, Lucian Li, Kieran Liu, Irvine Lu, Pony Ma, Rio Yang, Di Zhang, Adrian Zhou.

Team.Song Cao, Vic Cao, Kaijie Chen, Bunny Fan, Hera Feng, Huan Feng, Arthur Fu, Jun Gao, Hongquan Gu, Aaron Guan, Mutian Hong, Hailee Hou, Peixuan Hua, Charles Huang, Miles Jiang, Nora Jiang, Yuyi Jiang, Autumn Jin, Fancy Kong, Kyrie Lei, Alexy Li, Dawn Li, Ray Li, Theo Li, Jiayi Lin, Domini Liu, Heshan Liu, Kairus Liu, Logan Liu, Maeve Luo, Runism Lv, Pony Ma, Verity Niu, Anson Qiu, Vincent Wang, Maxwell Yao, Regis Ye, Wenlin Ye, Yanying Ye, Josh Ying, Danney Zeng, Salmon Zhan, Anya Zhang, Ruijia Zhang, Sueky Zhang, Ya Zhang, Wei Zhao, Ada Zhou, Sizer Zhou, Xinyue Zhu, Murphy Zhuang.

B Additional Serving Measurements

This appendix expands the serving-side measurements used in section 5.3. The tables use the same serving quantities as the main text: catalog size, CPU-cached adapters, adapters in one GPU batch, and the cold-load path that loads an adapter when a request selects an entry outside the actor's local cache. The order separates exploratory probes that rule out simple explanations, measurements that identify the serving limit, and follow-up probes that reduce or isolate that limit.

Several rows intentionally leave the clean warm path. They create weak locality, cold churn, long online generations, or high concurrency so that the failure mode is visible rather than hidden inside average latency. We keep them in the appendix because they identify the control points that a MinT deployment must own: routing for warm reuse, prewarming for predictable waves, bounded cold loading for cache-miss traffic, and lower-fanout adapter representation for unavoidable misses. The shaded bands in the tables separate cache levels and probe families, so a reader can scan from the MinT service path to the measured limit without treating every row as the same kind of claim.

Adapter memory and representation.Table 8 separates adapter bytes, tensor fanout, CPU cache footprint, and base-model HBM footprint. This accounting explains why the packed loader experiment matters. The measured adapter is moderate in byte size and fragmented into tens of thousands of small tensors, most of them no larger than 4 KB. Cold loading

therefore pays object and registration overhead even when the total bytes are small.

Tier / object	Item	Measurement	Serving implication
Adapter-file shape			
Adapter file	Adapter bytes	110.75 MB file; 105.5 MB declared tensor bytes	Bytes are moderate for a 30B rank-1 adapter.
Adapter file	Tensor fanout	37,248 tensors; 37,152 no larger than 4 KB	Object fanout dominates cold load work.
Resident execution footprint			
CPU warm tier	CPU cache footprint	586.8 MB/LoRA across 4 TP workers; 146.7 MB/worker	CPU cache is the per-engine cache limit.
GPU batch tier	GPU memory	$N = 64$ snapshots show about 272 GiB engine HBM footprint	Base deployment dominates HBM pressure.
Packed cold-load representation			
Adapter-file format	Packed representation	37,248 → 672 tensors; 110.75 → 105.58 MB	Packing removes fanout; bytes stay nearly unchanged.

TABLE 8 Memory and representation accounting for the measured 30B MoE LoRA adapter files. The rows separate byte size from object fanout and cache pressure.

Adapter catalog size.Table 9 varies the adapter catalog from 1k to 100k entries. The warm/cold split persists across the sweep and the measured tail appears when many distinct cold adapters enter cold loading. These rows support the main-text claim that MinT should keep catalog resolution in the control plane and manage cache state and cold loading in the serving plane.

Catalog	Regime	Success	Drain	p95	Distinct	Readout
1k	**warm**	64/64	21.16 s	20.89 s	51	catalog lookup stays in the warm regime
1k	**cold**	64/64	193.23 s	193.04 s	56	cold loading dominates
10k	**warm**	64/64	12.56 s	12.16 s	64	larger catalog stays in the warm mode
10k	**cold**	64/64	190.99 s	190.73 s	56	cold regime persists
100k	**warm**	64/64	20.35 s	12.12 s	63	100k-entry catalog remains addressable
100k	**cold**	63/64	190.08 s	189.73 s	56	tail follows cold loading

TABLE 9 Adapter catalog sweep for $N = 64$ serving. Warm and cold rows are split so the stable regime gap is visible at each catalog size.

Cached working sets.Table 10 gives the ordered data behind the warm-cache claim in figure 12. The repeated-hotset rows model adapter locality after routing has found a useful engine placement. The unique-adapter rows remove this locality and measure how many distinct adapters can become cached near one engine before the run stops being a clean warm-path claim. These measurements define the CPU-side tier between the durable adapter catalog and the same-batch adapter window.

Regime	Target	Loaded	Steady p95	Errors	Tier	Readout
Routed locality: repeated hotsets						
Repeated hotsets	128	127	5.52 s	0/3200	CPU warm	baseline warm tier
Repeated hotsets	192	183	10.59 s	0/3200	CPU warm	locality preserved
Repeated hotsets	256	236	20.74 s	0/3200	CPU warm	p95 rises with hotset
Repeated hotsets	384	317	30.12 s	0/3200	CPU warm	larger cached set
Repeated hotsets	512	369	37.13 s	0/3200	CPU warm	endpoint in figure
Unique-adapter calibration: clean active-window probes						
Unique adapters	16	16	1.22 s	0	GPU active	exact target
Unique adapters	32	32	1.34 s	0	GPU active	exact target
Unique adapters	64	64	1.41 s	0	GPU active	same-batch frontier
Unique-adapter ladder: pressure on local cache						
Unique adapters	128	127	5.70 s	0	CPU warm	loading begins
Unique adapters	192	183	10.57 s	0	CPU warm	mirrors hotset count
Unique adapters	256	236	17.79 s	0	CPU warm	cache growth
Unique adapters	384	317	38.37 s	0	CPU warm	tail appears
Unique adapters	512	369	45.36 s	0	CPU warm	warm cache still grows
Unique adapters	640	410	59.01 s	0	CPU warm	diminishing cache growth
Unique adapters	768	440	62.85 s	0	CPU warm	near plateau
Unique adapters	896	466	64.66 s	0	CPU warm	near plateau
Unique adapters	1024	486	66.92 s	0	CPU warm	mid-ladder endpoint
Unique-adapter high ladder: cache plateau						
Unique adapters	1280	513	60.91 s	0	CPU warm	plateau persists
Unique adapters	1536	525	60.95 s	0	CPU warm	slow cache growth
Unique adapters	1792	537	59.95 s	0	CPU warm	stable p95 band
Unique adapters	2048	550	63.14 s	0	CPU warm	largest measured target

TABLE 10 Cached-working-set ladder on one 4-GPU Qwen3-30B rank-1 serving actor. Shaded group rows separate routed locality from unique-adapter load pressure.

Mixed online-length stress traffic. Table 11 intentionally leaves the clean warm-path setting. These rows combine mixed output lengths, high concurrency, weak locality, prewarm changes, and different slot limits to show where simple routing choices stop being enough. The sticky-hash row is a negative probe: fixed hashing alone fails to provide cache-aware routing under this stress shape. The GPU-slots-64 row is a bounded probe; it verifies a slot-limit setting over two rounds, with long-run stability left outside this claim.

Axis	Sizing rule	Engines	GPUs	System interpretation
Warm-path placement				
Warm distinct concurrency	$\lceil 2300/64 \rceil$	36	144	ideal placement by same-batch adapter diversity
Warm headroom band	$36 \times \{1.2, 1.33, 1.5\}$	44–54	176–216	margin for skew, retries, and imperfect routing
Warm throughput floor	60 s SLO; 2.57 req/s/engine	15	60	throughput-only floor; ignores distinct placement
Cold-path isolation				
Cold-load rate	38.3 cold LoRA/s; 0.7/engine	55	220	rates the cold-load service with burst tail tracked separately
Cold-burst isolation	2300 cold uniques; $\leq$ 32/engine	72	288	isolates worst-case first touch through prewarm or backpressure

TABLE 13 Capacity-planning sketch from measured per-engine limits to a 1M-entry accumulated adapter catalog and a 2300-distinct-adapter active-wave stress envelope.

Probe	C	GPU/CPU	Prewarm	Success	Err.	p50	p95	p99	Interpretation
Single-route stress envelope									
Single route	8	32/1024	128	256/256	0.00%	28.79	66.23	70.47	low-concurrency baseline
Single route	16	32/1024	128	499/512	2.54%	63.18	134.03	179.74	tail begins to rise
Single route	32	32/1024	128	969/1024	5.37%	124.35	320.82	358.71	queueing and churn visible
Single route	128	32/1024	0	4032/4096	1.56%	508.92	1028.08	1403.44	high-concurrency stress
Single route	230	32/1024	0	7037/7360	4.39%	967.10	1469.37	1717.43	no-prewarm reference
Single route	230	32/2048	164	6707/7360	8.87%	926.94	1441.86	1710.01	slots alone leave tail high
Routing and slot probes									
Sticky hash, 2 replicas	230	32/2048	164	3737/7360	49.23%	925.62	1435.62	1746.58	naive fixed hashing unstable
GPU slots 64, 2 rounds	230	64/2048	164	460/460	0.00%	848.98	1351.34	1690.24	bounded slot-limit probe

TABLE 11 Mixed online-length traffic on a 2048-adapter catalog. The GPU/CPU column gives the GPU-batch LoRA slot limit and the CPU-cache LoRA limit.

ID	Probe	Measured result	Mechanism isolated	Design response
A1	Same-server no-warm, $N = 64$	Unique-no-warm p95 125.35 s; load median 60.27 s; queue wait median 0.010 s	LoRA loading drives waiting before generation; API queueing is small.	Separate cold loading from sampling latency.
A2	Same adapter vs unique adapters, $N = 16$	Same-adapter load median 1.56 s; unique adapters form 1.47, 2.84, 4.23, …, 23.83 s staircase	Repeated cache-miss requests can share one load.	Reduce unique cold misses through routing and prewarming.
A3	Structured add timing, $N = 16$	Engine time median 1.370 s; lock-wait median 10.890 s; total load p95 21.900 s	Exclusive writer/load path serializes unique loads.	Budget unique cold-load capacity per engine.
A4	Bounded cold-load probe	Max in-flight 1 and queue depth 1: a 4-request cold burst loads 2 and rejects 2	Cold pressure can be observable and retryable.	Prefer bounded backpressure over unbounded blocking.

TABLE 12 Cold-load accounting and service protection.

Cold-load accounting.Table 12 decomposes the cold path into API queueing, shared loads for repeated cache-miss requests, unique-adapter loading, and bounded backpressure. The probes show that the bottleneck is unique cold adapter loading into one engine. Requests wait before generation on LoRA loading; the measured API queue wait is small. Deduplicating identical missing adapters avoids repeated load work, while distinct cold adapters remain separate load jobs. This supports the main-text claim that cold loading is scheduled service work before ordinary sampling latency.

Fleet-level sizing sketch.Table 13 uses measured single-engine limits as sizing inputs for a larger MinT deployment. This table is a 10^6-adapter capacity model under a 2300-distinct-adapter active-wave assumption. The main serving experiments sweep adapter catalogs up to 100k entries. The extrapolation asks how many engines would be needed if a larger accumulated adapter catalog produced that active wave before routing or prewarming restored locality. The 60 s service-level objective is the warm-response target used for the throughput floor, and 2.57 req/s/engine is the observed warm-throughput input for that row. The cold-load rows size the separate case in which many selected adapters are cold at first touch.